

# Lesson 1: First Steps in Quantum Computing

## Lesson 1: First Steps in Quantum Computing From Classical Prediction to a One-Qubit Quantum Circuit

Dr. Syed Muzamil Basha

School of Computer Science and Engineering, REVA University

# Learning Outcomes

By the end of this lesson, a beginner should be able to:

- explain the difference between a classical bit and a qubit;
- describe the ideas of state, superposition and measurement at an introductory level;
- read a simple Qiskit quantum circuit;
- explain what the Hadamard gate does to  $|0\rangle$ ;
- interpret measurement counts produced by a simulator;
- connect a Python machine-learning example with the very different execution model of a quantum circuit.

# Why Start with a Classical Example?

## Classical machine learning example in the notebook

We first fit a straight line to a simple dataset:

$$y = 2x.$$

- Input values: 1, 2, 3, 4, 5
- Output values: 2, 4, 6, 8, 10
- The model learns a relationship from examples.
- The prediction for  $x = 8$  is 16.

**Key idea:** Classical programs process definite data values using ordinary bits, memory and arithmetic operations.

# Interpreting the Classical Python Code

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 6, 8, 10])

model = LinearRegression()
model.fit(X, y)

print(model.predict([[8]]))
```

- X: input feature values.
- y: target values.
- fit: estimates the best linear relationship.
- predict: applies the learned relationship to a new input.

# Interpreting the Classical Output

Expected output:

[16.]

- The training data follow the exact rule  $y = 2x$ .
- For the new input  $x = 8$ , the learned model returns 16.
- This is a deterministic prediction: the same input and same trained model produce the same result.

**Transition to quantum computing:** a quantum program can also be written with Python, but it manipulates quantum states rather than ordinary numerical features.

# Classical Bit vs Quantum Bit

| Concept                  | Classical bit      | Qubit                                           |
|--------------------------|--------------------|-------------------------------------------------|
| Basic states             | 0 or 1             | $ 0\rangle$ , $ 1\rangle$ , or a superposition  |
| State description        | Definite bit value | Quantum state with amplitudes                   |
| Operation                | Logic gate         | Quantum gate                                    |
| Readout                  | Bit value          | Measurement outcome                             |
| Typical beginner picture | Switch: OFF/ON     | Quantum state that can be prepared and measured |

A qubit is not simply a “faster bit”. It follows the rules of quantum mechanics.

# What Is a Qubit?

A single qubit can be written as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

where  $\alpha$  and  $\beta$  are probability amplitudes satisfying

$$|\alpha|^2 + |\beta|^2 = 1.$$

- $|0\rangle$  and  $|1\rangle$  are the computational basis states.
- Before measurement, the state may be a superposition of both basis states.
- Measurement produces a classical outcome: 0 or 1.

**Beginner interpretation:** amplitudes determine the probabilities we observe when the qubit is measured.

# The Hadamard Gate: The First Quantum Operation

The notebook applies a Hadamard gate with `qc.h(0)`.

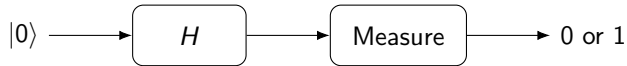
$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

$$H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

- Starting from  $|0\rangle$ , the Hadamard creates an equal superposition.
- The probabilities of measuring 0 and 1 are both 50% in the ideal case.
- The gate is reversible: applying  $H$  twice returns the original basis state.



# Read the Quantum Circuit as a Story



- 1 Create one qubit and one classical bit.
- 2 Put the qubit into an equal superposition with  $H$ .
- 3 Measure the qubit.
- 4 Store the classical measurement result.
- 5 Repeat or simulate many shots to observe the probability distribution.

# Interpreting the Quantum Python Code: Part 1

```
from qiskit import QuantumCircuit
from qiskit_aer import Aer

qc = QuantumCircuit(1, 1)
qc.h(0)
qc.measure(0, 0)
```

- `QuantumCircuit(1, 1)` creates 1 qubit and 1 classical bit.
- Qubit index 0 is the only qubit.
- `qc.h(0)` applies the Hadamard gate to qubit 0.
- `qc.measure(0,0)` measures qubit 0 and stores the result in classical bit 0.

## Interpreting the Quantum Python Code: Part 2

```
simulator = Aer.get_backend('aer_simulator')
job = simulator.run(qc)
result = job.result()
print(result.get_counts())
```

- `aer_simulator`: a local simulator for quantum circuits.
- `run(qc)` executes the circuit on the simulator.
- `job.result()` retrieves the completed result.
- `get_counts()` returns how many times each measured bit string occurred.

The current Qiskit documentation recommends installing Qiskit separately from Aer when local Aer simulation is needed. [?]

# Interpreting the Quantum Output

A typical result has the form

$$\{'0': 5xx, '1': 5xx\}$$

- The exact counts can vary from run to run because measurement is probabilistic.
- For an ideal Hadamard state, the two outcomes are expected to be close to equally frequent.
- With more shots, the observed proportions usually get closer to 50% and 50%.

$$P(0) \approx 0.5, \quad P(1) \approx 0.5.$$

**Important:** one measurement gives only one classical result; repeated shots reveal the underlying distribution.

# Why Is the Output Not Always Exactly 50–50?

Suppose 1024 shots are used. An ideal expectation is

$$E[N_0] = E[N_1] = 512.$$

But the actual counts might look like

$$0: \quad 496, \quad 1: \quad 528$$

or another nearby pair.

- Each shot samples the quantum state.
- Random sampling causes statistical fluctuation.
- Increasing the number of shots reduces the relative fluctuation.

**Analogy:** tossing a fair coin 100 times does not guarantee exactly 50 heads and 50 tails.

# What the Beginner Should Notice

## Classical example

The program learns a numerical relationship and returns a deterministic prediction for the given input.

## Quantum example

The program prepares a quantum state, applies a quantum operation, and measures the state to obtain classical data.

- Python is still the programming language.
- Qiskit gives us quantum-circuit abstractions.
- The important new ideas are qubits, gates, amplitudes, superposition and measurement.

# Beginner Hands-on Activity

Try these small changes in the notebook:

- ➊ Remove `qc.h(0)` and measure the initial state.
- ➋ Put `qc.h(0)` back and run many shots.
- ➌ Change the number of shots to 10, 100 and 1000.
- ➍ Compare the counts for 0 and 1.

## Prediction before execution:

- Without  $H$ : the qubit starts in  $|0\rangle$ , so measurement should return 0 ideally.
- With  $H$ : measurement should produce approximately equal numbers of 0 and 1 over many shots.

## Lesson 1 Summary

- A classical bit stores either 0 or 1.
- A qubit can exist in a superposition of basis states.
- The Hadamard gate creates an equal superposition from  $|0\rangle$ .
- Measurement converts a quantum state into a classical result.
- Repeated measurements are summarized as counts.
- Qiskit lets us construct and simulate quantum circuits using Python.

**Next step: build intuition with more quantum circuits and measurement experiments.**



## Quick Check

- ① What is the main difference between a bit and a qubit?
- ② What does the Hadamard gate do to  $|0\rangle$ ?
- ③ Why do the counts vary between repeated simulator runs?
- ④ What is the purpose of the classical bit in the circuit?
- ⑤ Why are multiple shots useful?

# Technical Note on the Notebook Environment

For a current local setup, IBM Quantum documentation recommends installing Qiskit in a virtual environment; Aer is installed separately when local Aer simulation is required. [?]

```
python -m venv .venv
# Windows PowerShell
.venv\\Scripts\\Activate.ps1
# Linux / macOS
source .venv/bin/activate
python -m pip install --upgrade pip
pip install qiskit qiskit-aer
```

The exact package versions should be taken from the environment used for the course rather than hard-coded into the lesson slides.



IBM Quantum Documentation, *Install Qiskit*, current Qiskit installation guide.

<https://quantum.cloud.ibm.com/docs/en/guides/install-qiskit>

This Lesson 1 deck is based on the uploaded Lesson1.ipynb notebook and expands its concepts for an audience with zero prior quantum-computing knowledge.